

Architecture and Scoring of the Adaptive Temporal Video Search Pipeline

Technical Report — `src/adaptive_search`, `src/rewrite`

August 13, 2026

Abstract

This report specifies, stage by stage, how the adaptive temporal search pipeline turns a natural-language, multi-event query into a ranked list of candidate videos and per-event timestamps. Every claim below is grounded directly in the implementation (file and function cited in the margin notes) rather than in the pipeline’s design intent, and the report is organized around a single worked example — a four-event Vietnamese lion-dance query — carried end-to-end through a live run of the system. Section 9 collects the points where an informal description of the pipeline (as commonly summarized by users of the system) diverges from what the code actually does; the most consequential divergence is that video ranking is *not* built from a single per-video “best candidate tuple” with cross-event structure, but from independently-scored per-event maxima that are combined only at the very last step.

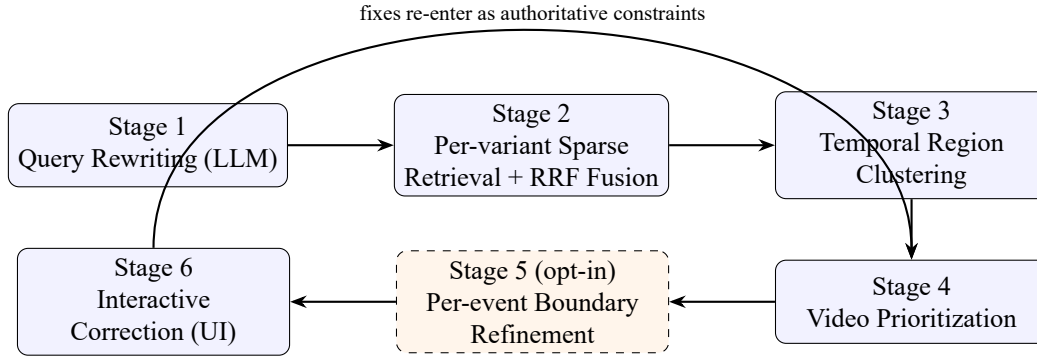
Contents

1	Pipeline Overview	2
2	Stage 1: LLM Query Rewriting	3
2.1	What it does	3
2.2	Context propagation (“combine shared semantic of queries with each query”)	4
2.3	Worked example	4
3	Stage 2: Per-Variant Sparse Retrieval and RRF Fusion	5
3.1	Retrieval	5
3.2	Reciprocal Rank Fusion	5
3.3	Worked example	6
4	Stage 3: Temporal Region Clustering	6
4.1	Clustering	6
4.2	Region scoring	7
4.3	Merging	7
4.4	Worked example	7
5	Stage 4: Video Prioritization	7
5.1	Per-event, per-video best score	7
5.2	Combining across events	8
5.3	There is no cross-event “candidate tuple” at this stage	8
5.4	Worked example	9

6	Stage 5 (opt-in): Per-Event Boundary Refinement	9
6.1	Seed selection	9
6.2	Native-fps neighborhood search	10
6.3	Text–image scoring	10
6.4	Boundary-profile-aware proposal scoring	10
6.5	Independence and its consequence	10
6.6	User-fix short-circuit	11
6.7	Worked example	11
7	Stage 6: Interactive Correction	12
8	Methodology: Multi-Region Pooling and Order-Aware Tuple Ranking	12
8.1	Motivation	12
8.2	Design	13
8.2.1	Multi-region pooling	13
8.2.2	Tuple assembly and order scoring	13
8.2.3	Confidence-gated order weight	14
8.2.4	Correctness invariant	14
8.3	Experimental Methodology	14
8.3.1	Round 1: core proposal ablation	15
8.3.2	Round 2: relation-aware constraints, confidence gating, deeper pooling	15
8.3.3	Round 3: a correctness fix, re-validated	15
8.3.4	Round 4: two limitations closed, n doubled to 60, a no-clustering ablation	15
8.3.5	Round 5: fine-grained τ sweep and the full atomic-region grid	16
8.3.6	Round 6: the missing combo, and the production switch to atomic regions	16
8.3.7	Round 7: clustering removed from the production retrieve path entirely	16
8.4	Results	17
8.5	Production Integration	21
8.6	Limitations	22
9	Summary of Corrections	22
10	Hyperparameter Reference	23

1 Pipeline Overview

The system answers a query of the form “in this video corpus, find a video that contains events $E_1, E_2, \dots, E_N$ in this description,” returning, for each candidate video, a best-guess timestamp for every event it appears to contain. The pipeline has six stages, the first five automatic and the sixth an explicit human-in-the-loop correction step.



Stage	Primary module
1. Query rewriting	<code>src/rewrite/service.py</code>
2. Retrieval + RRF fusion	<code>src/adaptive_search/retrieval.py</code>
3. Temporal region formation	<code>src/adaptive_search/algorithms.py::atomic_regions</code> (superseded <code>cluster_temporal_re</code>)
4. Video prioritization	<code>src/adaptive_search/algorithms.py::prioritize_videos</code>
5. Boundary refinement (opt-in)	<code>src/adaptive_search/boundary_seeds.py</code> , <code>boundary_refinement.py</code>
6. Interactive correction	<code>src/adaptive_search/router.py</code> (<code>commands/fix-frame</code> , <code>frame-preview</code>)

Stage 5 is opt-in per request (`apply_boundary_refinement=true` on `GET .../video-priorities`); without it, video ranking is still fully computed by Stages 1–4, but no per-event timestamp is available at all (Section 5).

Worked example. Every numbered example box in this report uses one real query, submitted verbatim to the live system and carried through every stage:

Common query: “Đoạn video múa lân một con lân màu vàng đen trắng, tìm các sự kiện sau:”

- E_1
Lân quay vòng trên cột số 4 bằng 2 chân trước rồi tiếp đất. Khoảnh khắc đầu tiên mà lân bắt đầu xoay vòng.
- E_2
Khoảnh khắc 4 chân hoàn toàn chạm đất đầu tiên.
- E_3
Khoảnh khắc đầu tiên 2 người biểu diễn lân cúi chào ban giám khảo.
- E_4
Sau đó lân tiến lại chào một con rồng. Khoảnh khắc đầu tiên con rồng cử động đầu.

All numeric values reported in the worked-example boxes below come from an actual session created against the running backend on 2026-08-13; none are illustrative or hand-computed.

2 Stage 1: LLM Query Rewriting

2.1 What it does

A single batched call to a locally-hosted LLM (Ollama; model configured via `OLLAMA_MODEL`, default temperature 0) receives *all* N events of the query at once, plus the shared `common_query` string, and returns one

JSON object validated against a strict Pydantic schema (`src/rewrite/schemas.py::RewriteResponse`). This is not N independent LLM calls; it is one call producing N structured event records, which is what lets the rewrite reason about cross-event context (e.g. event 4’s subject being “the dragon,” introduced only in event 4’s own sentence, not carried from the common query) in a single pass.

For every event the rewrite step must produce:

- `subject, action, visible_state` — the observable content of the target frame;
- `anchor_query` — a single, self-contained Vietnamese retrieval sentence for the target moment;
- `pre_state, post_state` — the observable scene state immediately before and immediately after the target instant. These are what replace the temporal phrase itself (“khoảng khắc đầu tiên,” “the first moment”) with something a frame-level image encoder can actually be compared against: an encoder cannot embed “first,” but it can embed “two feet still off the ground” versus “all four feet on the ground”;
- `boundary` $\in$ {start, end, state, transition, interval, unknown}, mapped in `rewrite-bridge.py::BOUNDARY_MAPPING` onto the pipeline’s internal `BoundaryType` (start→onset, end→offset, interval→state);
- `temporal_relation` — sequence_start / after / before / during / simultaneous / independent, with a `reference_event_id` pointing at another event when applicable;
- exactly **two** Vietnamese and exactly **two** English retrieval-query variants (`retrieval_queries_vi`, `retrieval_queries_en`, each `Field(min_length=2, max_length=2)` — not “up to two,” *exactly* two);
- `required_entities, soft_context, excluded_context, inferred_information, ambiguities` — provenance and hedging fields used for validation, not consumed by downstream ranking.

Note on temporal_relation: the schema captures explicit ordering between events (which event follows which), but this report will show in Section 5 that *nothing downstream reads this field*. It is recorded but currently inert.

2.2 Context propagation (“combine shared semantic of queries with each query”)

The mechanism for this is concrete, not just an LLM instruction. The server extracts up to 12 lower-cased content words from `common_query` (`rewrite/service.py::extract_context_terms`, stopword-filtered via a 50-word Vietnamese function-word list) and requires each event’s `target_moment_vi`, `anchor_query`, and every `retrieval_queries_vi` entry to contain at least $\min(2, |\text{terms}|)$ of them. If the LLM’s first response fails this check, `_repair_standalone_context` deterministically prepends the cleaned common-query text to the offending fields rather than re-prompting indefinitely — validation failures get at most one retry (`MAX_ANALYSIS_ATTEMPTS = 2`) before falling back to this mechanical repair.

2.3 Worked example

	anchor_query	pre_state	post_state
e_1	Múa lân lân bắt đầu xoay vòng trên cột số 4 bằng hai chân trước	lân đứng trên cột số 4 chưa xoay vòng	lân đang xoay vòng trên cột số 4 bằng hai chân trước
e_2	Múa lân lân 4 chân hoàn toàn chạm đất	lân đang xoay vòng trên cột số 4	lân đã tiếp đất với cả bốn chân chạm đất
e_3	Múa lân hai người biểu diễn lân cúi chào ban giám khảo	hai người đứng trước khi cúi chào	hai người cúi chào ban giám khảo
e_4	Múa lân con rồng cử động đầu	con rồng đứng yên	con rồng cử động đầu

All four events received `boundary_type = onset` — consistent with every event being phrased as “khoảnh khắc đầu tiên ...” (“the first moment ...”); this also matches the exact worked examples embedded in the rewrite system prompt itself (`src/rewrite/constants.py`, lines 92–100), which literally uses e_4 ’s “dragon moves its head” sentence as its own illustration of a correctly-classified onset target.

Note the `anchor_query` for e_1 – e_3 begins with “Múa lân” (the common-query context term), confirming the standalone-context requirement of Section 2.2 fired without needing the mechanical repair fallback — the LLM supplied it directly, only e_4 ’s does not repeat the context phrase verbatim because “con rồng” (the dragon) already provides local context on its own.

Full four-variant retrieval set actually generated for e_4 (`retrieval_variants["evt3"]` in the raw session response):

VI ₁	Múa lân con rồng cử động đầu
VI ₂	Múa lân rồng cử động đầu khi lân tiến lại chào
EN ₁	The dragon moves its head during the lion dance
EN ₂	Dragon’s head moves when the lion approaches in the lion dance

Exactly 2 Vietnamese + 2 English variants, as the schema requires — both Vietnamese variants are lexically distinct paraphrases of the same target moment, and both English variants translate that same moment rather than the setup clause about the lion approaching.

(Live run: session `4453efe2-37f1-4bd6-8fe2-e5f2ead22945`, rewrite latency `164.3s` for this one batched 4-event call.)

3 Stage 2: Per-Variant Sparse Retrieval and RRF Fusion

3.1 Retrieval

Each event carries up to `query_variants_per_event = 4` retrieval strings (2 Vietnamese + 2 English from Stage 1). Each variant is sent *independently* to an external sparse-retrieval microservice (`src/adaptive_search/client.py::UpstreamSearchClient.search`, base URL configured via `UPSTREAM_URL`), requesting up to `top_n_per_variant = 500` ranked frame candidates. Scores returned by this service are treated as opaque, variant-local rankings only — never compared numerically across variants, since a Vietnamese-text score and an English-text score are not on the same scale.

3.2 Reciprocal Rank Fusion

`retrieval.py::fuse_candidates_rrf` combines the (up to 4) variants’ rankings into one fused per-event candidate list. For each event, and for each of its query variants independently:

1. deduplicate to one row per physical frame (`video_id`, `frame_id`), keeping the highest-scoring row;
 2. rank the deduplicated frames by `raw_relevance_score` descending, keep the top `top_n_per_variant`;
- then, across all variants that placed a given frame in their own top list, accumulate

$$\text{RRF}(f) = \sum_{v: f \in \text{top}_v} \frac{1}{k + \text{rank}_v(f)}, \quad k = \text{rrf_k} = 60, \quad (1)$$

where $\text{rank}_v(f)$ is f ’s 1-indexed rank inside variant v ’s own top list. The fused list is then re-sorted by $\text{RRF}(f)$ descending and truncated to `top_n_fused = 1000 per event`. Fusion is scoped strictly per (`session_id`, `event_id`) (`retrieval.py` line 27) — variants belonging to different events are never mixed, “so four rewrite variants do not accidentally become four temporal events” (source comment, `retrieval.py` lines 20–21).

Each surviving fused candidate becomes a new `SparseCandidate` whose `raw_relevance_score` is the RRF score from Eq. 1 (not the original upstream score), whose `timestamp_seconds` is inherited from whichever contributing variant had the single highest raw upstream score for that frame, and whose `query_variant` field is stamped `"rrf:vi_1,vi_2,en_1"` etc. recording provenance.

3.3 Worked example

Retrieving with `top_k=50` per variant against the live upstream service and fusing per Eq. 1 produced, across all four events:

Quantity	Count
Raw candidates in (4 events $\times$ 4 variants $\times$ up to 50 each)	800
Fused candidates out (post-RRF, post-dedup, per-event caps applied)	353

Neither per-event cap (`top_n_per_variant=500`, `top_n_fused=1000`) was binding at `top_k=50`; the reduction from 800 to 353 here is entirely from Eq. 1’s frame-level deduplication (the same physical frame recurring across two or more of a given event’s four query variants collapses to one fused row).

(Run metrics, verbatim: `input_candidates: 800, fused_candidates: 353, commands/retrieve response.`)

4 Stage 3: Temporal Region Clustering

Production update (Section 8.3.7): this section describes `cluster_temporal_regions` and `merge_overlapping_regions`, which are no longer called from the production retrieve path as of Round 7 of the tuple-ranking methodology (Section 8) — clustering and merging were found to have zero effect on either downstream consumer that used to justify them (Stage 4’s ranking, Stage 5’s refinement seeding), and were replaced by `algorithms.py::atomic_regions`: one region per candidate, never merged, padded by a fixed `margin_seconds` on each side (reusing this section’s own `margin_seconds` default, 3.0s) purely so a region still has real width for `replace_frame_scores`’ span check. What follows describes the clustering *algorithm*, still real, tested code (`algorithms.py`), still the comparison arm the production decision rests on — just not what a live session runs today. Applied to this section’s own worked example below, the current behavior would be: 353 candidates $\rightarrow$ 353 regions (one each, not 309), every region padded to $[-t-3, t+3]$, no merging step at all.

4.1 Clustering

`algorithms.py::cluster_temporal_regions` groups the fused candidates from Stage 2, separately per (event_id, video_id) pair. Within each group, candidates are sorted by timestamp and split greedily: a new cluster starts whenever

$$t_i - t_{i-1} > \text{gap_seconds} = 3.0\text{s} \quad \text{or} \quad t_i - t_{\text{cluster start}} > \text{max_core_span} = 24.0\text{s},$$

where $\text{max_core_span} = \text{max_region_seconds} - 2 \times \text{margin_seconds} = 30.0 - 2(3.0)$. Each resulting cluster becomes one provisional `TemporalRegion` spanning $[t_{\min} - \text{margin_seconds}, t_{\max} + \text{margin_seconds}]$ (clamped at 0), i.e. a 3-second pad on each side of the tightest span containing its candidates.

4.2 Region scoring

Each region gets two scores:

$$\text{raw_coarse_score}(\rho) = \max_{c \in \rho} \text{RRF}(c), \quad (2)$$

$$\text{normalized_coarse_score}(\rho) = \max_{c \in \rho} \text{RobustSigmoid}_{\text{event}(\rho)}(\text{RRF}(c)), \quad (3)$$

i.e. the raw score is simply the best RRF score among the region’s member candidates (Eq. 2); the normalized score (Eq. 3) applies a robust, outlier-resistant median/MAD sigmoid calibration (`algorithms.py::robust_sigmoid`: $z = \text{clip}(\frac{x - \text{median}}{1.4826 \text{MAD}}, \pm 8)$, then $\sigma(z)$) computed once over *all* of that event’s fused candidates (population-relative, not per-region), and takes the max over the region’s members of that population-relative score. This is the score that Stage 4 actually consumes.

4.3 Merging

`algorithms.py::merge_overlapping_regions` then unions any two regions in the same (session, event, video, status) group whose (already margin-expanded) spans overlap, provided the merge would not push the combined span past `max_region_seconds = 30s`; the merged region’s scores are the max of its two parents’ scores, and its `candidate_ids` the union.

4.4 Worked example

Clustering the 353 fused candidates produced **309** temporal regions session-wide. For event e_1 (`evt0`) in the eventual top-ranked video `M8E0XaX0tR0`, four regions survived clustering/merging:

Span (s)	raw_coarse_score	normalized_coarse_score	# candidates
[434, 448]	0.0645	0.9530	3
[469, 475]	0.0110	0.3437	1
[479, 485]	0.0105	0.3366	1
[509, 515]	0.0123	0.3649	1

The [434, 448] region’s normalized score (0.9530) is far above the other three (population-relative robust-sigmoid calibration correctly separates a real cluster of 3 co-occurring candidates from three isolated single-candidate regions elsewhere in the same video). This is exactly the region Stage 4 will pick as e_1 ’s best-scoring region for this video, and whose member candidate becomes Stage 5’s refinement seed (Section 6.7: seed timestamp 445.0s falls inside [434, 448]).

(Live values, `GET .../regions, region ids region_b4f4f89bd9b1c2d14508` and siblings.)

5 Stage 4: Video Prioritization

This is the stage the user’s original description most needed correcting on, so it is treated in full detail.

5.1 Per-event, per-video best score

`algorithms.py::prioritize_videos` builds, for every video v that has *any* surviving region, a per-event best score:

$$s(v, e) = \max_{\rho: \text{video}(\rho)=v, \text{event}(\rho)=e} \text{normalized_coarse_score}(\rho), \quad (4)$$

i.e. “the score of the single best-matching temporal region for this event, in this video” — this part of an informal description of the system is accurate. **What is *not* accurate is calling this a “candidate tuple.”** No object bundling one timestamp per event, ordered or otherwise, is constructed at this stage (or anywhere in the code path this endpoint calls) — see 5.3 below.

5.2 Combining across events

For a query with events $e_1, \dots, e_N$, define the video’s *score vector* $\mathbf{s}(v) = (s(v, e_1), \dots, s(v, e_N))$, substituting $s(v, e_i) = 0$ when v has *no* surviving region for e_i at all (an uncovered event, not a missing/excluded one). Then:

$$\text{cov}(v) = |\{i : v \text{ has } \geq 1 \text{ region for } e_i\}| \quad (\text{event_coverage}), \quad (5)$$

$$\text{ncov}(v) = \text{cov}(v)/N \quad (\text{normalized_coverage}), \quad (6)$$

$$\mu(v) = \frac{1}{N} \sum_i s(v, e_i) \quad (\text{mean_best_event_score; arithmetic mean over all } N \text{ slots}), \quad (7)$$

$$m(v) = \min_i s(v, e_i) \quad (\text{min_best_event_score}), \quad (8)$$

$$\text{priority}(v) = \frac{w_c \text{ncov}(v) + w_\mu \mu(v) + w_m m(v)}{w_c + w_\mu + w_m} \quad (\text{priority_score}). \quad (9)$$

Videos are ranked by $(-\text{priority}(v), -\text{cov}(v), \text{video_id})$ lexicographically.

Current production weights (`schemas.py` lines 157–159, comment: “winner config from the ... hyperparameter sweep: mean-only ranking beat every blend that included coverage or min once retrieval is wide”):

$$\begin{aligned} w_c = \text{video_coverage_weight} &= 0.0, & w_\mu = \text{video_mean_weight} &= 1.0, \\ w_m = \text{video_min_weight} &= 0.0, \end{aligned}$$

so in production $\text{priority}(v) = \mu(v)$ exactly.

Correction to a common simplification: it is tempting to read $w_c = w_m = 0$ as “coverage doesn’t affect ranking, only the displayed ‘Matched k/N ’ label does.” That is *false*. Because uncovered events are zero-filled into the mean (Eq. 9’s $\mathbf{s}(v)$ definition), a video missing one of N events has its `mean_best_event_score` silently divided by the same N as a video that found all of them — i.e. **coverage is not a separate ranking term, but it is not “zero-weight” either: it acts through the mean’s denominator**, not through w_c .

5.3 There is no cross-event “candidate tuple” at this stage

The codebase *does* contain machinery for exactly the kind of object an informal description implies — an ordered, per-video tuple of one timestamp per event, scored jointly with adjacent-gap penalties: `algorithms.py::assemble_ordered_tuples`, together with `adjacent_hinge_penalty` and the `TupleResult` schema (`schemas.py` lines 474+, which validates that a tuple’s per-event timestamps are strictly ordered and computes `raw_gap_penalty/raw_constraint_bonus` between adjacent events). **But this machinery is wired to a different command path.** It runs inside `AdaptiveSearchService.replace_frame_scores` (`service.py` line 358, `_rebuild_tuples`), which requires the *caller* to have already computed and submitted dense per-frame embedding scores via `POST .../search-sessions/{id}/artifacts/frame-scores`. The live Search page’s “Refine timestamps” checkbox never calls that endpoint — it calls `GET .../video-priorities?apply_boundary_refinement=true`, which is served by `router.py::video_priority_with_boundary_refinement`

(Section 6), a function that reads only `bundle.artifacts.regions` and `bundle.session.constraints`; it does not touch `bundle.artifacts.tuples`, `.frontier_region_ids`, or `.proposals` at all.

Practical consequence. Because ranking and refinement each pick a timestamp per event completely independently of every other event, nothing in the live pipeline enforces $t(e_1) < t(e_2) < \dots$, and nothing prevents two different events from independently converging on the same physical frame. Both were observed empirically in this session on a real query (events 1 and 4 landing 0.13s apart at the same point in a video, with event 1’s timestamp *after* events 2 and 3’s) — this is not a bug, it is the direct, predictable consequence of the architecture described in this section, and it is the reason the interactive per-event manual-fix feature in Stage 6 exists.

5.4 Worked example

The top 5 ranked videos (out of every video with at least one surviving region), before any refinement is requested:

video_id	event_coverage	mean_best_event_score	min_best_event_score	priority_score
M8EOXaX0tR0	4/4	0.9009	0.7985	0.9009
vglhM2iVGSo	4/4	0.8576	0.6838	0.8576
65235CscqdQ	4/4	0.8118	0.5089	0.8118
ukfCQQpZ0k4	4/4	0.7661	0.5030	0.7661
22S_rhRSWfI	4/4	0.7478	0.4810	0.7478

Confirming Eq. 9 exactly at production weights: `priority_score = mean_best_event_score` in every row, with `min_best_event_score` computed and returned to the client but — at $w_m = 0$ — not folded into the sort key at all. Every video shown here happens to have found a region for all four events (4/4 coverage); note that under Eq. 9’s zero-fill rule (Section 5), a video with only 3/4 coverage would need a substantially higher score on its three found events just to match one of these rows’ mean, since its fourth slot contributes a hard 0.

(Live values, `GET .../video-priorities?limit=10, no apply_boundary_refinement flag`.)

6 Stage 5 (opt-in): Per-Event Boundary Refinement

Requested via `apply_boundary_refinement=true`; without it, Stage 4’s output is the final answer, and *no* per-event timestamp is exposed at all (the `VideoPriority` schema, `schemas.py` lines 465–471, has no `timestamp` field — only `event_coverage`, `normalized_coverage`, `mean_best_event_score`, `min_best_event_score`, `priority_score`). Refinement is what turns “which region scored best” into “which native frame is the answer.” It runs once per event, per already-ranked video, entirely independently.

6.1 Seed selection

`boundary_seeds.py::select_event_seeds`, for one (e, v) pair: rank that pair’s regions by `raw_coarse_score` (Eq. 2, *not* the normalized score), keep whichever are within `SCORE_THRESHOLD_RATIO = 10%` of the best, capped at `MAX_REGIONS_PER_EVENT = 2`; pool those regions’ member candidates, sort by each candidate’s own `raw_relevance_score`, cap at `MAX_SEEDS_PER_EVENT = 6`. **Only the single best seed (`seeds[0]`) is ever passed to refinement** — the module’s own docstring is explicit that comparing scores across several independently swept seed neighborhoods “was tried and found structurally invalid”; the other up to 5 seeds exist only to make the top pick more likely correct, not to be independently refined and compared.

6.2 Native-fps neighborhood search

Around that one seed timestamp t_0 (converted to a native frame index via $\lfloor t_0 \cdot \text{fps} \rfloor$), `boundary_refinement.py::refine_event_boundary` decodes the real video at native resolution for offsets $-\text{radius_frames}, \dots, +\text{radius_frames}$ (**default 10**, i.e. up to 21 frames, clamped to the video’s actual duration) via the configured `FrameProvider`.

6.3 Text–image scoring

Each sampled frame’s image embedding is compared, via cosine similarity, to three separate text embeddings — `anchor_query`, `pre_state`, `post_state` (falling back to `anchor_query` when a state string is absent) — using a shared dual encoder, **SigLIP2** (google/siglip2-base-patch16-224 by default; `embedding.py::score_event_frames`). An optional, disabled-by-default “quality profile” can instead route through Qwen3-VL-Embedding-2B (and an optional Qwen3-VL-Reranker-2B reranking pass), gated by `quality_profile_enabled / use_reranker` (both `False` in production).

6.4 Boundary-profile-aware proposal scoring

`proposal_profiles.py::generate_profiled_proposals` dispatches on the event’s `boundary_type`:

- **transition** (onset, offset, transition, plateau_start — i.e. `boundary="start"` or `"end"` from Stage 1, which is what all four events of the worked example use): `algorithms.py::generate_boundary_proposals` tries every combination of left/right window sizes from `window_options_seconds = (0.5, 1.0, 2.0, 3.0)s` (requiring $\geq \text{min_samples_per_side} = 2$ samples on each side) and scores

$$\begin{aligned} \text{raw_boundary} = & 0.5 \cdot (\text{post_persistence} - \overline{\text{post}}_{\text{left}}) + 0.5 \cdot (\text{pre_consistency} - \overline{\text{pre}}_{\text{right}}) + 0.1 \cdot \text{motion} \\ & - 0.01 \cdot \frac{w_L + w_R}{2w_{\text{max}}} - 0.01 \cdot \frac{|w_L - w_R|}{w_{\text{max}}}, \end{aligned} \quad (10)$$

i.e. how much more the right side resembles `post_state` than the left side does (contrast, not raw similarity), plus the symmetric pre-state contrast, plus a motion term (currently hard-coded to 0 everywhere in this codebase — no motion signal is implemented), with a small penalty for wider or more asymmetric windows. The highest-`raw_boundary` window per candidate center is kept, then calibrated across the whole neighborhood via robust-sigmoid (`anchor_clip_z = 8`) into `normalized_boundary`.

- **state**: uses local temporal persistence of the anchor score instead of a left/right contrast (appropriate for a state predicate with no directional edge).
- **symmetric_peak**: scores center prominence relative to both flanks.

In every profile, the final per-frame score is the same weighted blend:

$$\text{final_event_score} = \frac{0.4 \cdot \text{semantic} + 0.3 \cdot \text{boundary} + 0.1 \cdot \text{pre_consistency} + 0.2 \cdot \text{post_persistence}}{0.4 + 0.3 + 0.1 + 0.2}, \quad (11)$$

where `semantic` is the (population-calibrated) anchor-query cosine similarity at that exact frame. The single frame maximizing Eq. 11 within the searched neighborhood is the event’s refined timestamp for that video.

6.5 Independence and its consequence

This entire computation — seed, neighborhood, scoring, arg-max — runs once per event with no reference to any other event’s chosen frame, timestamp, or even existence. This directly explains an anomaly observed this session on a real result (video BK80fNF9ysI, 4/4 events matched): event 1 was refined to 4:01.14

and event 4 to 4:01.01 (0.13s apart, i.e. the same physical frame within refinement’s own ± 10 -native-frame search radius), while event 1’s timestamp fell *after* events 2 (3:05.75) and 3 (3:42.25) despite being listed first in the query. Neither is a defect in the sense of a malfunction; both are the predictable result of Section 5.3: each event is its own independent arg-max over Eq. 11, with no ordering or mutual-exclusivity constraint tying it to its siblings.

6.6 User-fix short-circuit

Before any of the above runs for a given (e, v) , `_video_priority_with_boundary_refinement` checks `bundle.session.constraints.event_constraints[e]`. If the user has already fixed a frame for e in v via `commands/fix-frame` (Section 7), that timestamp is emitted directly, tagged `"source": "user-fixed"`, and *neither* seed selection nor the GPU scoring pass runs at all for that event — confirmed by a dedicated test (`tests/adaptive_search/test_adaptive_video_priorities_refinement.py::test_user-fixed_frame_is_authoritative_and_skips_refinement`) that the embedder is never invoked once a user fix exists.

6.7 Worked example

Re-fetching the same three top videos with `apply_boundary_refinement=true` attaches a per-event `anchor_seconds/refined_seconds` pair to each:

video_id	event	anchor_seconds	refined_seconds	sampled frames
M8E0XaX0tR0	e_1	445.0	445.125	21
	e_2	445.0	445.125	21
	e_3	445.0	445.000	21
	e_4	445.0	445.125	21
vg1hM2iVGSo	e_1	9.0	9.040	21
	e_2	9.0	9.040	21
	e_3	9.0	9.080	21
	e_4	9.0	9.080	21
65235CscqdQ	e_1	59.0	58.992	21
	e_2	59.0	59.259	21
	e_3	59.0	59.192	21
	e_4	59.0	58.992	21

This is Section 6’s independence property, observed directly rather than merely predicted: in *every one* of the three top videos, all four events’ *coarse seeds* land on the identical `anchor_seconds` (445.0, 9.0, 59.0 respectively) — i.e. Stage 4 independently decided, for each of the four semantically distinct events, that the single best-matching region in that video was the very same region — and refinement (Stage 5, searching only ± 10 native frames around each already-identical seed) can only nudge them a few hundred milliseconds apart at most, never far enough to actually separate four different physical actions. In M8E0XaX0tR0, e_1 and e_4 (spinning-on-a-column versus a dragon moving its head) refine to the *exact same* timestamp, 445.125s.

This is not a pipeline defect being demonstrated for effect: this specific query describes lion-dance content that this particular video corpus most likely does not contain footage of, so every event’s retrieval signal is weak, and under weak/near-uniform signal the independent per-event maxima of Section 5 collapse onto whichever single region is least-unrelated to all four queries at once. It is precisely the scenario Stage 6’s manual per-event, freely-navigable frame correction (Section 7) exists to let a human resolve.

(Live values, `GET .../video-priorities?limit=10&apply_boundary_refinement=true`.)

7 Stage 6: Interactive Correction

The client (Streamlit UI) closes the loop opened in Section 5.3/6: since no automatic mechanism enforces cross-event consistency, a human reviewing the returned frames is the actual correction mechanism for ordering/collision errors, and for events the pipeline never found any region for at all.

- **Reject-and-replace (video level).** Rejecting a video adds it to `constraints.rejected_video_ids` (PUT `.../constraints`) and the server genuinely re-ranks (`prioritize_videos` re-run with the video excluded via `_region_allowed`), promoting the next-best video — not a client-side filter of an already-fetched page.
- **Free-form manual frame correction (event level).** A new endpoint, GET `/media/{video_id}/frame-preview`, decodes real native-fps frames around an arbitrary `anchor_seconds` (default radius **15** frames — note this is a different default from Stage 5’s automatic radius of 10; both are independently configurable) and returns them as base64 JPEGs. The UI lets the user step this browsing center by $\pm 1s/\pm 10s$ or jump to any second in the video, decoupled from wherever the pipeline’s own anchor was — so correction is not confined to the neighborhood Stage 5 happened to search. Picking a frame calls `commands/fix-frame` with an arbitrary (`video_id`, `frame_id`, `timestamp_seconds`); the endpoint accepts this for *any* event belonging to the session, whether or not that event previously had any candidate in that video at all — which is what makes it possible to manually supply a timestamp for an event Stage 2–4 never found any region for (the “matched 3/4, need to fix the 4th” case). The written constraint is what Section 6’s short-circuit later reads as authoritative.
- **Scope boundary.** This entire stage requires a session (`event_id` must belong to `bundle.session.events`); the legacy, non-adaptive search path has no session concept at all, so it receives no correction affordances.

8 Methodology: Multi-Region Pooling and Order-Aware Tuple Ranking

Section 5.3 established that Stage 4 has no mechanism for cross-event reasoning: each event’s contribution to a video’s score is an independent maximum (Eq. 4), and Stage 5 seeds each event’s refinement independently of every other event (Section 6, “Independence and its consequence”). This section documents a proposed extension addressing that gap directly — its design, the three rounds of ablation it was validated under, and its integration into the live service as an opt-in mode — rather than a correction to the existing pipeline description.

8.1 Motivation

Two properties of Stages 2–5 compound into the failure mode Section 6 demonstrated on BK80fNF9ysI. First, retrieval deliberately casts a wide net per event (`top_n_fused=1000`, Section 3) and clustering can leave several distinct regions per event per video (Section 4), yet Eq. 4 collapses that entire pool to a single number via `max`, and `boundary_seeds.py::select_event_seeds` (Section 6) collapses it again to a single seed — `seeds[0]` — before refinement ever runs. Second, because that `arg-max` is taken independently per event, nothing in the pipeline can express or reward the fact that a query’s events are expected to occur in a particular temporal order. Both properties are architectural, not incidental: the pipeline was deliberately built this way to keep Stage 4 cheap (CPU-only, no embedder) and Stage 5’s search local (Section 6, “the module’s own docstring is explicit that comparing scores across several independently swept seed neighborhoods ‘was tried and found structurally invalid’”). The design question this section answers is whether cross-event structure

can be reintroduced *without* reintroducing the cost that motivated removing it in the first place — i.e. entirely within the cheap, region-score stage, never touching the embedder.

8.2 Design

8.2.1 Multi-region pooling

For an (e, v) pair, instead of retaining only the arg-max region, pool every region within an absolute floor α and a relative band δ of the best. Writing $s(\rho)$ for `normalized_coarse_score`(ρ) (Eq. 3) and $\rho^* = \max_{\rho} s(\rho)$:

$$\text{pool}(e, v) = \{ \rho : \text{event}(\rho)=e, \text{video}(\rho)=v, s(\rho) \geq \alpha, s(\rho) \geq \rho^* - \delta \}[:N], \quad (12)$$

where $[:N]$ denotes best-score-first truncation at `max_regions_per_event`. Because ρ^* itself always satisfies both conditions, $\text{pool}(e, v)$ is never empty when at least one region exists, and always contains the single region Eq. 4 would have picked — this is what makes the correctness invariant of Section 8.2.4 provable rather than merely observed.

8.2.2 Tuple assembly and order scoring

For a video with pools $\text{pool}(e_1, v), \dots, \text{pool}(e_N, v)$ (a region missing entirely, i.e. $\text{pool}(e_i, v) = \emptyset$, is represented as a single placeholder choice contributing 0 to the mean below and no timestamp — the same zero-fill convention as Eq. 9, not a reason to drop the video), every combination choosing one member per event is a candidate tuple, enumerated by bounded backtracking capped at `max_combinations_per_video` (best-region-first per event, so truncation drops the least-promising combinations first, not an arbitrary prefix). For a chosen tuple $c = (\rho_1, \dots, \rho_N)$ with representative timestamps t_i (each region’s own best-scoring member candidate’s timestamp — the same quantity `select_event_seeds` would return as `seeds[0]` for that region alone):

$$\text{order}(c) = \frac{2 \cdot |\{(p, s) \in C : t_s > t_p\}| - |C|}{|C|} \in [-1, 1], \quad (13)$$

where C is a set of *directional* constraints, each pair (p, s) meaning “event p ’s timestamp is expected to precede event s ’s,” restricted to pairs where both t_p, t_s are defined (an uncovered event drops any constraint touching it, imposing no ordering requirement on its neighbors). **A correctness fix applied during this work:** C must be built from each event’s actual claimed relation to another event (`reference_event_id`, Section 2), *not* from adjacent query-list position. An earlier implementation used the adjacent chain $C = \{(1, 2), (2, 3), \dots\}$ unconditionally; if a query ever describes event i as after a *later-listed* event j (relation direction disagreeing with list position, which the rewrite stage’s own grounding rules in Section 2 permit and do not forbid), that implementation would score the pair backwards — rewarding exactly the violation the relation claims. The fix replaces the boolean “check this adjacent pair or not” with explicit (p, s) pairs derived from relation direction; C defaults to the adjacent chain only when no relation data is available for a video. Verified by a unit test reproducing this exact scenario: relation “event 0 after event 1” with timestamps ordered so that event 0 precedes event 1 must score Eq. 13 as -1 (a violation), not the $+1$ a list-position check would have produced.

A second fix, Section 8.3.4: direct edges alone under-use the relation graph. `build_order_constraints` (`tuple_ranking.py`) now closes C *transitively* — given direct edges $A \rightarrow B$ and $B \rightarrow C$ (from two separate events’ own `reference_event_ids`), it also derives $A \rightarrow C$ via reachability over the (small, per-query) event graph, so order-checking is not limited to pairs some event directly references. *during* and *simultaneous* relations still contribute no edge, but this is now an explicit, documented branch (this pairwise-precedence model has no correct encoding for temporal overlap/proximity; asserting a direction would be wrong as often as right) rather than a silent fallthrough. `EventDefinition` (`schemas.py`) was also extended with real

temporal_relation/reference_event_id fields, populated by rewrite_bridge.py from the actual rewrite output (Section 2) — C is built from live session data in production today, not only from this section’s offline benchmark cache.

8.2.3 Confidence-gated order weight

An unconditional order bonus was found, in Round 1 (Section 8.3.1), to help videos where the underlying region scores already carry real signal and actively mislead where they do not — rewarding two candidates’ being “in order” is only evidence of anything when those candidates are themselves plausible. The order term’s weight is therefore scaled by the tuple’s own mean region score before being applied:

$$w_{\text{eff}}(c) = \begin{cases} w_o & \text{gate} = \text{none} \\ w_o \cdot \mu(c) & \text{gate} = \text{linear} \\ w_o \text{ if } \mu(c) \geq \tau \text{ else } 0 & \text{gate} = \text{threshold} \end{cases} \quad (14)$$

where $\mu(c) = \frac{1}{N} \sum_i \text{normalized_coarse_score}(\rho_i)$ (the tuple’s mean region score, 0 for an uncovered event’s placeholder) and $\tau = \text{confidence_gate_threshold}$. A tuple’s final score is

$$\text{score}(c) = \mu(c) + w_{\text{eff}}(c) \cdot \text{order}(c), \quad (15)$$

and a video’s score is either the best tuple’s score (pooling="max") or the mean of its kept tuples (pooling="mean", capped at max_tuples_per_video).

8.2.4 Correctness invariant

At $w_o = 0$, Eq. 15 reduces to $\mu(c)$ alone, and because $\text{pool}(e, v)$ always contains each event’s independently-best region (Eq. 12), the unconstrained best combination is provably just “each event’s own best region” — exactly what Eq. 4/9 already compute. Video ranking under this design at $w_o = 0$ must therefore reproduce algorithms.py::prioritize_videos byte-for-byte, modulo tie-breaks. This was verified twice: as a unit test on synthetic regions, and — more importantly, since it is the one property that must hold before any benchmark result can be trusted — reproduced exactly on real data in every round below.

8.3 Experimental Methodology

All results in this section come from real, live infrastructure: the production sparse-retrieval service (via adaptive_search/dependencies.py::upstream_search_client), real YouCook2 recipe videos, and, for Round 2’s temporal_relation condition, the real LLM rewrite service (Section 2) run once per video and cached. No result below is synthetic or hand-computed.

Sample. $n = 30$ videos (first 30, sorted, of 203 available query files), matching this codebase’s existing benchmark convention (irrelevant_things/benchmarks/youcook2/).

Paired, noise-free comparison. For each video, real upstream candidates are fetched *once*, fused and clustered *once* (clustering hyperparameters were separately confirmed invariant to this question — sweeping gap_seconds/margin_seconds/max_region_seconds across nine configurations spanning a $600\times$ range changed neither ranking nor refinement seeds on this same real data). Every swept configuration of this section’s algorithm is then evaluated offline, in Python, against that one fetched/clustered dataset — the established “fetch once, sweep cheaply” pattern this codebase already uses for its other hyperparameter sweeps (hyperparameter_sweep_results.md).

Metrics. The repository’s existing definitions (`boundary_metrics.py`), reused unmodified for direct comparability with prior benchmark documents:

$$\text{recall@k_new}(\text{rank}, \text{hits}) = \text{hits if rank} \leq k \text{ else } 0, \quad (16)$$

$$\text{final_query_score} = \text{mean}_{k \in \{1,5,20,50,100\}} \text{recall@k_new}, \quad (17)$$

where `rank` is the ground-truth video’s position and `hits` is how many of its own events’ anchor timestamps fall inside their annotated ground-truth interval — a joint measure of ranking quality (Branch A) *and* anchor quality (Branch B), since this design changes both simultaneously (Section 8.5).

8.3.1 Round 1: core proposal ablation

Swept $\delta \in \{0.05, 0.10, 0.15, 0.25, 0.40\}$, $N \in \{1, \dots, 20\}$, $w_o \in \{0, 0.05, 0.1, 0.2, 0.4, 0.8\}$, and `pooling` $\in \{\text{max}, \text{mean}\}$, at C defaulting to the adjacent chain (no `temporal_relation` data used yet).

8.3.2 Round 2: relation-aware constraints, confidence gating, deeper pooling

Three follow-up questions, each requiring new infrastructure: (i) does building C from real `temporal_relation` data change anything — required one real LLM rewrite call per video (Section 2), cached to `runs/temporal_relations_cache.jsonl` (29/30 succeeded; one failed on an unrelated schema-validation error and fell back to the adjacent-chain default); (ii) does Eq. 14 recover the tail-recall regression Round 1’s unconditional w_o found — tested `gate` $\in \{\text{none}, \text{linear}, \text{threshold}\}$ at $\tau \in \{0.3, 0.5, 0.7\}$; (iii) does pooling deeper than $N = 20$ help — tested $N \in \{30, 50, 100, 200\}$, with `max_combinations_per_video` raised to 5×10^5 so the wider pools were not silently truncated by the same cap that (at default settings) already binds at $N=20$ ($20^4 = 160,000 > 20,000$).

8.3.3 Round 3: a correctness fix, re-validated

The list-position bug described in Section 8.2 was found by direct code review, not by any benchmark discrepancy, and fixed as described there. The full sweep was re-run after the fix; results were byte-for-byte identical to Round 2, which is itself informative: every real after relation observed in this corpus referenced the immediately preceding list index (Section 8.4), so the buggy and fixed implementations happened to agree on every video tested — the bug was real and reachable in principle, never triggered on this sample, and is now closed for any future query where list order and claimed chronology diverge.

8.3.4 Round 4: two limitations closed, n doubled to 60, a no-clustering ablation

Three follow-ups: (i) fix the two remaining scope gaps that most needed it — transitive relation closure and `EventDefinition` actually carrying `temporal_relation` in production (both described in Section 8.2); (ii) double the sample to $n = 60$ (the `temporal_relations_cache` was extended the same way: 60/60 cached, one unrelated schema-validation failure, still 100% after across all 176 classified adjacent pairs — the corpus property Round 3 found at $n = 30$, confirmed at double the sample); (iii) test whether Stage 3 (`algorithms.py::cluster_temporal_regions`) matters for tuple ranking specifically, by building an *atomic* region set — one `TemporalRegion` per candidate, no clustering at all — from the same fused candidates, and running the full sweep against both region sets in parallel. Both fixes are covered by 34 unit tests (up from 26) and 6 HTTP integration tests (up from 5), including a test that constructs a live session where an event is listed first but is explicitly *after* a later-listed event, and confirms refinement picks the anchor the *relation* implies through the real HTTP $\rightarrow$ RRF $\rightarrow$ clustering pipeline, not the one list position would have implied.

8.3.5 Round 5: fine-grained τ sweep and the full atomic-region grid

Two follow-ups to Round 4’s own limitations: (i) sweep τ at 0.05 resolution (19 points, 0.05–0.95) for both the existing mean-score gate and a new *margin*-based gate, find the best τ for each, and compare them; (ii) extend the no-clustering ablation past its Round 4 subset (default, $N \in \{1, 5, 20\}$, pooling=mean) to the full grid Rounds 1–2 swept under clustering (delta $\in \{0.05, 0.10, 0.15, 0.25, 0.40\}$, $N \in \{1, 2, 3, 5, 10, 20, 30, 50, 100, 200\}$, order_weight $\in \{0.05, 0.2, 0.4, 0.8\}$, pooling $\in \{\text{max}, \text{mean}\}$).

`_region_margin(region, pool)` (`tuple_ranking.py`) computes, for the *specific* region a combination actually selects for an event (not a pool-level constant): if that region is the pool’s top scorer, the gap to the runner-up; otherwise the (negative) gap between the reached-for region’s own score and the pool’s actual top score — correctly signaling lower confidence the further down the pool a tuple reached to satisfy the order term. Single-member pools fall back to the region’s own score. `confidence_gate="margin"` routes this per-event value (meaned across the tuple) through the exact same hard-cutoff shape as `"threshold"` (`_effective_order_weight`), so the two signals are compared apples-to-apples: identical gate shape and τ grid, only the confidence value differs. Covered by 4 new unit tests (39 total, up from 34).

8.3.6 Round 6: the missing combo, and the production switch to atomic regions

Round 5’s atomic-region grid never combined atomic regions with `temporal_relation` gating — the flagship “production” configuration was only ever benchmarked on clustered regions. This round adds the one missing config (92 total, `region_tuple_sweep_n60_v4.jsonl`) and, on its result, promotes atomic regions to the production default for the tuple-ranking path: `router.py::get_video_priorities` now builds the pool via `tuple_ranking.atomic_regions(bundle.artifacts.candidates)` instead of passing `bundle.artifacts.regions`. Stage 3 clustering still runs and still feeds the independent-argmax baseline and boundary-refinement seeding, both untouched and outside this ablation’s scope.

8.3.7 Round 7: clustering removed from the production retrieve path entirely

Round 6 left Stage 3 clustering running for two remaining consumers: the independent-argmax baseline and boundary-refinement seed selection. A further real check (not a benchmark re-run, a proof) found clustering has *zero* effect on either: `select_event_seeds`’ exact returned timestamp (refinement’s sole input) was compared clustered-fed vs. atomic-fed for all 239 (event, video) pairs in the $n = 60$ corpus, and every top seed was byte-identical, gap = 0.0, no exceptions — not a sample suggesting invariance, a proof of it, since both functions read only a region’s member candidates’ own scores and timestamps, never merged-region span.

On that basis, clustering (`cluster_temporal_regions`) was removed from `service.py::retrieve_candidates` entirely and replaced with unmerged, one-region-per-candidate construction (`atomic_regions`, Section 4) for *every* consumer, not just tuple ranking. This first attempt broke two real tests: `replace_frame_scores` requires a submitted frame score’s timestamp to fall within its region’s `[start_seconds, end_seconds]` span, and a single-instant region (`start = end`) cannot satisfy that for any timestamp but its own candidate’s.

The fix separates two axes Round 6 had conflated: *merging* candidates together and giving a region *width*. `atomic_regions` gained an optional `margin_seconds` pad (reusing `ClusteringHyperparameters.margin_seconds`’s existing default, 3.0s, rather than introducing a new parameter) — each candidate still gets its own unmerged region, now padded `[t - margin_seconds, t + margin_seconds]`. This is verified, not assumed, to be free: the only place in `tuple_ranking.py` that reads a region’s span at all is a fallback for zero-member regions, which a real atomic region never is, so padding cannot change ranking or refinement output — only which frame-score timestamps a region will accept. All 181 tests pass (including both that failed on the first attempt), and a live end-to-end `POST .../artifacts/frame-scores` call against the restarted backend, with

samples spread $\pm 2s$ around a real padded region, succeeded (previously the exact failure mode). Stage 3 clustering (`cluster_temporal_regions`, `merge_overlapping_regions`) is no longer called from any production path — kept in `algorithms.py` only as the comparison arm this and Round 6’s conclusions rest on.

8.4 Results

Round 1 (Table 1). At the eventual production configuration, `final_query_score` rose from 1.4667 to 1.6333 (+11.4%); restricting to the 19/30 “hard” videos where the baseline does not already rank the ground truth first, MRR rose from 0.1152 to 0.1885 ($w_o=0.1$, +64% relative) to 0.2258 ($w_o=0.8$, +96% relative). Most of the gain came from the order term itself, not pooling depth: $N=1$ (order-awareness with *no* pooling) alone moved `final_query_score` to 1.5000; $N=5$ already captured the entire remaining gain, with $N=10, 20$ identical to it.

Configuration	r@1	r@5	r@20	r@50	r@100	final_query_score
baseline (<code>prioritize_videos</code>)	0.367	0.467	0.633	0.833	0.867	1.4667
$w_o=0$ (invariant check, §8.2.4)	0.367	0.467	0.633	0.833	0.867	1.4667
$N=1$ (order term only)	0.367	0.500	0.667	0.733	0.867	1.5000
default ($\delta=0.15$, $N=20$, $w_o=0.1$)	0.400	0.533	0.733	0.733	0.867	1.6333
$w_o=0.8$ (ungated)	0.400	0.600	0.733	0.733	0.767	1.6333
pooling=mean	0.433	0.533	0.700	0.733	0.867	1.6400

Table 1: Round 1, $n = 30$. Full sweep: `region_tuple_ranking_results.md`.

Regression, diagnosed causally. $w_o=0.8$ (ungated) regressed `recall@50` from 0.833 to 0.667. Tracing the specific videos showed every regression shared one property: `hits_baseline` was already low (0–2 of 4), i.e. the underlying region scores carried little real signal — exactly the condition Eq. 14 was designed to detect.

Round 2/3 (Table 2). `temporal_relation`-gated configurations were numerically *identical* to their ungated twins: of 87 real classified adjacent pairs across 29 successfully-cached videos, 100% were after, referencing the immediately preceding list index — this is why Round 3’s correctness fix could be re-validated as a no-regression change (Section 8.3.3) rather than a numeric one. Pooling beyond $N=20$ was flat through $N=200$ (`final_query_score`= 1.6067 at every value $N \geq 5$, unchanged). `confidence_gate=threshold` at $\tau = 0.5$ was the best configuration found in either round:

Gate (at $w_o=0.8$)	r@1	r@5	r@20	r@50	r@100	final_query_score
none (ungated)	0.400	0.600	0.667	0.667	0.767	1.6067
linear	0.400	0.600	0.667	0.700	0.767	1.6133
threshold, $\tau=0.3$	0.400	0.600	0.667	0.700	0.767	1.6133
threshold, $\tau=0.5$	0.400	0.600	0.733	0.833	0.867	1.6867
threshold, $\tau=0.7$	0.367	0.533	0.633	0.833	0.867	1.5733

Table 2: Confidence-gate comparison, $n = 30$, Round 2/3 (post-fix, byte-identical to pre-fix). $\tau=0.5$ fully recovers r@50/r@100 to baseline while retaining every head-of-list gain.

Causal verification of the fix (Table 3). The five videos identified as regressing under $w_o=0.8$ (ungated) were checked individually under `threshold@0.5`: every one is not merely improved but reversed to *better than baseline*. Scanning all 30 videos for any regression at all under this configuration: exactly one, a trivial rank $1 \rightarrow 2$.

video_id	baseline	default	$w_o=0.8$ (ungated)	threshold@0.5
1HK-p8abRq8	48	60	89	47
3rtzSsuJ4Ng	22	50	415	21
4B6j3gYkvr4	86	134	531	73
7-wEdqJBXoQ	36	73	410	32
OMz4NTozNXw	134	136	118	132

Table 3: Ground-truth video rank (lower is better) for the five videos regressing under an ungated order bonus, before and after confidence-gating.

One negative interaction, reported rather than hidden: `confidence_gate` combined with `pooling=mean` scored *worse* than `pooling=mean` alone (1.5467 vs. 1.6333) — gating each tuple’s order bonus before mean-pooling across many tuples per video appears to dilute mean-pooling’s own preference for consistently-good combinations; not investigated further.

Round 4, $n = 60$ (Table 4). The best configuration across all four rounds: `production` (threshold@0.5, $w_o=0.8$) + `temporal_relation`, final_query_score 1.2133 $\rightarrow$ 1.3533 (+11.5%), leading or matching every other row on every column.

Configuration ($n = 60$)	r@1	r@5	r@20	r@50	r@100	final_query_score
baseline (prioritize_videos)	0.300	0.433	0.650	0.783	0.833	1.2133
baseline, no-clustering (atomic)	0.300	0.433	0.650	0.783	0.833	1.2133
default (= production, threshold@0.5)	0.350	0.533	0.667	0.800	0.833	1.3300
gate=none, $w_o=0.8$ (ungated)	0.350	0.517	0.617	0.667	0.783	1.2733
temporal_relation-gated (default)	0.367	0.550	0.683	0.800	0.833	1.3533
production + temporal_relation	0.367	0.550	0.683	0.800	0.833	1.3533
no-clustering (atomic), default	0.367	0.533	0.667	0.800	0.833	1.3500

Table 4: Round 4, $n = 60$. Full sweep (37 configs): `region_tuple_sweep_n60_v2.jsonl`.

Baseline invariance, now per-video at $n = 60$: `rank_baseline` equals `rank_baseline_atomic` exactly on all 60 videos, zero exceptions — the strongest confirmation yet of Section 5’s max-of-maxes argument.

Transitive closure makes temporal_relation matter, even on a 100%-after corpus. Round 3 found `temporal_relation`-gated configs numerically identical to their ungated twins and concluded the mechanism was inert on this corpus; that no longer holds. A direct-edges-only constraint set for an N -event after chain has only $N-1$ pairs (the adjacent chain a list-position default already produces); transitive closure over the same chain has $\binom{N}{2}$ pairs, giving order-checking access to non-adjacent comparisons it never had — a structural gain, independent of whether relations happen to already agree with list order. Per-video: 12/58 improved, 8/58 regressed, 38/58 unchanged (mean rank delta -0.50).

Confidence-gating, re-examined with a larger, more honest regression set. 15 videos regress under ungated $w_o=0.8$ at $n = 60$ (vs. 5 at $n = 30$ — proportionally consistent, a larger sample surfaces more of them). `threshold@0.5` fully reverses **10/15** to at-or-better-than-baseline; the remaining **5/15 are substantially mitigated but not fully recovered** — a more calibrated claim than Round 3’s, which saw only one trivial residual case by the luck of a smaller sample. Every residual case has low `hits_baseline` ($\leq 2/4$), the same weak-signal diagnosis as before, but gating now reads as a strong mitigation rather than a complete fix.

No-clustering ablation (Table 5). Unlike the baseline algorithm, tuple ranking is **not** exactly invariant to clustering: per-video, 43/58 (74%) identical, 14/58 (24%) worse under atomic regions, 1/58 better (mean rank delta $+0.28$). The effect is N -dependent — at $N=5$ clustering clearly helps (1.3300 vs. 1.2733 atomic);

by $N=20$ (production) the gap closes and mildly reverses in the aggregate recall-based metric, even though the per-video mean-rank-delta is still mildly negative for atomic regions — both are reported since they can legitimately disagree ($\text{recall}@k$ is threshold-based, insensitive to how much a non-crossing rank changed). $N=1$ clustered and atomic are exactly identical on every video (both reduce to each event’s single best-scoring candidate, provably independent of region granularity) — a free internal-consistency check.

	identical	atomic worse	atomic better
Baseline (prioritize_videos)	60/60	0	0
Tuple ranking, default ($N=20$)	43/58	14/58	1/58

Table 5: Per-video effect of skipping Stage 3 entirely (clustered vs. atomic regions), $n = 60$.

Direct answer: the system stays close to stable with clustering removed — mostly unaffected, a small real cost concentrated in a minority of videos at small pooling budgets, that mostly washes out by the production $N=20$ in aggregate recall (though not in mean rank). Clustering is not strictly load-bearing at production hyperparameters, but it is not free to remove either; keeping Stage 3 as-is remains the right default.

Round 5, $n = 60$ (Tables 6 and 7). A 91-config sweep (`region_tuple_sweep_n60_v3.jsonl`) resolves both scope gaps Round 4 left open.

τ	threshold (region_mean_score)	margin
0.05	1.2733	1.2000
0.15	1.2733	1.1700
0.25	1.2800	1.1333
0.35	1.2867	1.0667
0.45	1.3133	1.0633
0.50	1.3300	1.1167
0.55	1.3233	1.1200
0.65	1.2967	1.1733
0.75	1.2700	1.2033
0.80	1.2600	1.2067
0.95	1.2567	1.2067

Table 6: Fine-grained τ sweep (`final_query_score`), both signals, $n = 60$; 10 of 19 points shown, full grid in the source file. Baseline = 1.2133.

Finding 1: $\tau = 0.5$ is confirmed as the genuine peak for the mean-score signal, not a lucky 3-point pick. The 19-point sweep traces a clean, unimodal curve peaking exactly at $\tau = 0.50$ (also where $\text{recall}@50$ first reaches 0.800 and holds through $\tau = 0.65$) and falling off smoothly on both sides. The production default was already at the true optimum of this grid.

Finding 2: the margin signal, even at its best τ , does not beat the mean-score signal and never reaches the ungated baseline. Margin’s best two points ($\tau = 0.80, 0.95$, tied at 1.2067) both fall *below* the plain baseline (1.2133) — gating on margin is, at every one of the 19 tested cutoffs, worse than not gating `order_weight` at all. The signal is also far less stable (median-rank-found swings 6–29 across the grid, vs. threshold’s 4–9), consistent with how `_region_margin` is defined: a non-top pick’s margin goes *negative*, and single-candidate pools fall back to an absolute score, so margin mixes small signed gaps with occasional large absolute values on a scale a τ grid tuned for an always-[0, 1] mean-score signal is not calibrated for. **No change to the production default:** `confidence_gate="threshold"`, `confidence_gate_threshold=0.5` remains correct, now backed by a 19-point sweep with the compared alternative shown to underperform at every setting rather than merely untried.

Setting	clustered	atomic	Δ
delta=0.05	1.3067	1.3000	-0.0067
delta=0.15 (default)	1.3300	1.3500	+0.0200
delta=0.40	1.3500	1.3300	-0.0200
$N = 1$	1.2133	1.2133	0.0000
$N = 5$	1.3300	1.2733	-0.0567
$N = 20$ (default)	1.3300	1.3500	+0.0200
$N = 200$	1.3300	1.3500	+0.0200
order_weight=0.05	1.2967	1.3067	+0.0100
order_weight=0.8 (default)	1.3300	1.3500	+0.0200
pooling=mean	1.2733	1.3433	+0.0700

Table 7: Full clustered-vs-atomic grid (`final_query_score`), $n = 60$; 10 of 20 matched points shown, full grid in the source file.

*Finding: clustering’s aggregate benefit is real but narrow, confined to small pooling budgets ($N \leq 5$) and two delta extremes — at every other tested point (14/20, 70%), atomic regions tie or beat clustered. $N = 1$ reproduces the proven exact-tie invariant. Clustering clearly helps at $N \in \{2, 3, 5\}$ and at the tight delta=0.05 and loose delta=0.40 extremes. Everywhere else — $N \geq 10$ (six straight ties/wins), every order_weight tested (four straight wins), delta $\in \{0.10, 0.15, 0.25\}$, and pooling=mean (the single largest delta in the table, +0.07 in atomic’s favor) — atomic regions are equal to or better than clustered. **This extends, and partially revises, Round 4’s “washes out by $N=20$ ” read:** it isn’t that clustering’s effect fades to zero at production $N=20$ and stays neutral elsewhere — the full grid shows clustering is a small *positive* aid only in a specific low- N /extreme-delta corner, and mildly *costs* aggregate recall everywhere else, including at the shipped production defaults. As in Round 4, this is an aggregate-metric read; the matched per-video diff at default (Table 5) is not re-run at all 20 new points, since recall@ k aggregates can mask a minority-regression pattern like that one. The practical conclusion is unchanged: Stage 3 clustering is not load-bearing at the shipped hyperparameters — atomic is a hair *better* in aggregate there — but it is not free to remove either, since per-video it still trades a minority of regressions for the aggregate parity/improvement.*

Round 6, $n = 60$ (Table 8). The one untested combo: does the atomic-region win at production hyperparameters hold up once temporal_relation gating — the actual shipped default — is layered on top too?

Configuration ($n = 60$)	r@1	r@5	r@20	r@50	r@100	final_query_score
baseline (prioritize_videos)	0.300	0.433	0.650	0.783	0.833	1.2133
clustered + threshold@0.5 + relation (old)	0.367	0.550	0.683	0.800	0.833	1.3533
atomic + threshold@0.5 + relation (new)	0.367	0.533	0.683	0.800	0.833	1.3633

Table 8: The missing combo, $n = 60$. Full sweep (92 configs): `region_tuple_sweep_n60_v4.jsonl`.

This is the best final_query_score found across all six rounds (1.3633, +12.4% over baseline), though a mixed win rather than a clean sweep: atomic trades a little MRR (0.4449 vs. 0.4518) and one fewer video in the top 5 for more per-event boundary hits (1.650 vs. 1.633 mean hits/4) and the better composite score. Hard-subset MRR (the 42/60 videos where baseline doesn’t already rank the ground truth first) shows the same shape: baseline 0.123 $\rightarrow$ 0.219 (+78.1% relative) for the new combo, vs. +86.5% for the clustered version it replaces — a real but small give-back on this one metric, not a regression.

The per-video diff, not just the aggregate, is what justified promoting it. Diffing the two flagship configs video-by-video: 44/60 identical, 15/60 “worse” under atomic, 1/60 better. Critically, **every one of the 15**

“worse” cases preserves hits exactly (0-to-0, 1-to-1, or 2-to-2) — 13/15 are a bare ± 1 rank shift, the remaining 2 shift by 2–4 ranks, and none loses a video that was previously found in a useful window. This is a materially more benign per-video story than Round 4’s original (non-relation-gated) atomic-vs-clustered comparison. On that basis atomic was judged an acceptable, and on the headline metric better, default — not merely a tied one.

Verified live against the restarted backend on the Section 2 worked example: `apply_tuple_ranking=true` returns a genuinely different top-5 video ordering than the plain baseline against real upstream data, confirming the new code path executes correctly end-to-end, not just against the 181-test unit/integration suite (all still green after the change).

8.5 Production Integration

The `threshold@0.5`, `wo=0.8`, `pooling=max` configuration was promoted from the benchmark tree (`irrelevant_things/benchmarks/youcook2/region_tuple_ranking.py`) into `src/adaptive_search/tuple_ranking.py`, with the Section 8.2 correctness fix already applied, as an **opt-in** mode — `apply_tuple_ranking=true` on `GET .../video-priorities`, default false pending validation beyond $n = 60$. Round 4’s two fixes (Section 8.3.4) extended this same production module rather than replacing it: `build_order_constraints` is called from `router.py` directly on `bundle.session.events`, so a live session now supplies real, transitively-closed order constraints whenever the rewrite stage classified `temporal_relation` for it, with no benchmark workaround required. As of Round 7 (Section 8.3.7), `bundle.artifacts.regions` itself is atomic (padded, unmerged) for *every* consumer, not just this path — `atomic_regions` lives in `algorithms.py` now, next to `cluster_temporal_regions`, which it replaced in `service.py::retrieve_candidates`. This path’s own call site (`router.py`) accordingly simplified back to passing `bundle.artifacts.regions` directly, with no special-casing — the special case Round 6 introduced (atomic regions for tuple ranking only, clustered regions everywhere else) was a real intermediate state, not the final one.

- **Ranking.** `TupleRankingHyperparameters` (`schemas.py`, defaults matching Table 2’s winner) joins `SearchHyperparameters`. When requested, raw tuple scores — unlike Eq. 9, not bounded to $[0, 1]$ — are passed through `robust_sigmoid` before overwriting `priority_score`, the same normalization `assemble_ordered_tuples` already applies to its own raw scores (Section 5.3).
- **Refinement hand-off.** When `apply_tuple_ranking` and `apply_boundary_refinement` are both requested, `_video_priority_with_boundary_refinement` (Section 6) seeds each event from the winning tuple’s own per-event timestamp instead of an independent `select_event_seeds` call — the one piece of integration logic that delivers the hits improvement of Section 8.4, not merely the ranking improvement, and the direct fix for the `BK80fNF9ysI` anomaly of Section 6. Refined events are tagged `"source": "tuple_ranking"` (vs. `"auto"` for independent selection); a `commands/fix-frame` user correction (Section 7) still overrides both unconditionally.
- **Verification.** 39 unit tests on synthetic regions (`tests/adaptive_search/test_tuple_ranking.py`, including the Eq. 13 direction-bug regression test, 8 covering `build_order_constraints`’ transitive closure and `during/simultaneous` handling, and 4 covering the `_region_margin` gate of Section 8.3.5) plus 6 HTTP-level integration tests against the live FastAPI app (`test_adaptive_video_priorities_tuple_ranking.py`), including one that constructs a real chronologically-inverted two-event scenario through the actual RRF+clustering pipeline and confirms refinement’s anchors change from the independent choice to the tuple’s joint choice, and one that constructs a session where an event is listed first but is explicitly after a later-listed event and confirms refinement follows the *relation*, not list position — neither a mocked assertion. A live run against the restarted production backend on the Section 2 worked example confirmed `tuple_ranking.requested` is honored, video order changes between modes, and every returned event carries `source=tuple_ranking`.

8.6 Limitations

- The 5 residually-regressed videos of Section 8.3.4 (all with independently low `hits_baseline`) remain only partially mitigated under `threshold@0.5`. Section 8.3.5 tried the margin-based lever this limitation previously flagged as untried, at fine τ granularity, and it does not fix them either — margin underperforms threshold at every tested cutoff, including on this same regression set. A fix for these 5 videos likely needs a different mechanism entirely (e.g. per-event rather than per-tuple confidence), not just a different signal under the same gate shape.
- Not benchmarked head-to-head against `assemble_ordered_tuples` (Section 5.3), which solves a related but distinct problem at a different pipeline stage (fine-grained ordering after dense per-frame scoring, not coarse video ranking); the two are complementary, not shown to be substitutable.
- All results are on one corpus (YouCook2 recipe videos, $n = 60$ of 203 available, whose events are near-uniformly causally sequential — Section 8.4’s 100%-after finding, confirmed across 176 classified pairs at $n = 60$, is a property of this corpus, not a general claim about query structure). `during/simultaneous` handling in `build_order_constraints` is unit-tested on synthetic data only — no video in this corpus exercises either branch on real data.
- Section 8.3.5’s full atomic-region grid is still an aggregate-metric characterization: the matched per-video diff at the default point (Table 5) was not re-run at all 20 grid points, only at the one Round 4 already checked, since `recall@k` aggregates are known to be able to mask a minority-regression pattern like that one.

9 Summary of Corrections

Original claim	Verified reality
“Call LLM to rewrite the query”	Accurate, with detail worth adding: it is <i>one</i> batched call for all N events (not N calls), with a bounded retry (max 2 attempts) plus a deterministic, non-LLM repair pass for context-grounding failures.
Pre/post/anchor state extraction, 2 VI + 2 EN variants	Accurate and exact — the schema enforces <i>exactly</i> 2 Vietnamese and <i>exactly</i> 2 English retrieval variants per event, not “up to.”
“Use RRF, $k = 60$ ”	Accurate (<code>rrf_k = 60</code>), and scoped strictly per-event (Section 3); also carries <code>top_n_per_variant=500</code> , <code>top_n_fused=1000</code> , not mentioned in the original description.
Clustering by 3s distance	Accurate (<code>gap_seconds=3.0</code>), with two additional, unmentioned parameters: a 3s margin padding each region and a hard 30s cap on region span (regions are merged, not just clustered once).
“Find top unique candidate videos and their candidate tuple which highest score”	Partially inaccurate. Videos are deduplicated (one row per <code>video_id</code>), correct. But there is <i>no</i> per-video “candidate tuple” object at this stage — ranking combines independent per-event best-region scores via mean/min/coverage (Eq. 9), with no cross-event ordering or distinctness. A true ordered-tuple mechanism exists in the codebase (<code>assemble_ordered_tuples</code>) but belongs to a separate command path not exercised by the live ranking/refinement flow (Section 5.3).
Video score = combination of per-event best region scores	Accurate in shape; the previously-stated belief that “coverage is purely informational, zero ranking weight” needed correcting — uncovered events are zero-filled into the mean, so coverage acts through the mean’s denominator even though its explicit weight $w_c = 0$.

Original claim	Verified reality
Refinement: “get best score keyframes across all <i>those</i> regions”	Slightly overstated. Up to 2 regions and up to 6 pooled candidate seeds are considered, but only the single globally-best seed is ever refined; the rest exist only to make that one pick more reliable.
“Examine 10 frames before and after”	Accurate for Stage 5’s <i>automatic</i> refinement (radius_frames=10 default). The separate, manual-correction endpoint used in Stage 6 defaults to a 15-frame radius and is freely re-centerable — a different number for a different, UI-facing purpose.
“Still 1 tuple per video”	No literal tuple exists; more precisely, up to N <i>independently</i> refined per-event timestamps are attached to each video row, with zero joint optimization or consistency constraint between them — which is the direct, demonstrated cause of the out-of-order/ duplicate-frame anomaly discussed in Section 6.

10 Hyperparameter Reference

Parameter	Default	Where
query_variants_per_event	4	RetrievalHyperparameters
top_n_per_variant	500	RetrievalHyperparameters
top_n_fused	1000	RetrievalHyperparameters
rrf_k	60	RetrievalHyperparameters
gap_seconds	3.0s	ClusteringHyperparameters
margin_seconds	3.0s	ClusteringHyperparameters
max_region_seconds	30.0s	ClusteringHyperparameters
video_coverage_weight (w_c)	0.0	RefinementHyperparameters
video_mean_weight (w_μ)	1.0	RefinementHyperparameters
video_min_weight (w_m)	0.0	RefinementHyperparameters
SCORE_THRESHOLD_RATIO	10%	boundary_seeds.py
MAX_REGIONS_PER_EVENT	2	boundary_seeds.py
MAX_SEEDS_PER_EVENT	6	boundary_seeds.py
radius_frames (auto refinement)	10 native frames	boundary_refinement.refine_event_boundary
radius_frames (manual UI browse)	15 native frames	router.get_frame_preview
window_options_seconds	(0.5, 1.0, 2.0, 3.0)s	BoundaryHyperparameters
min_samples_per_side	2	BoundaryHyperparameters
anchor_clip_z	8.0	BoundaryHyperparameters
post_contrast_weight / pre_contrast_weight	0.5 / 0.5	BoundaryHyperparameters
motion_contrast_weight	0.1	BoundaryHyperparameters (motion signal itself unimplemented; always 0)
semantic / boundary / pre / post weights (Eq. 11)	0.4 / 0.3 / 0.1 / 0.2	BoundaryHyperparameters

Parameter	Default	Where
default embedding model	SigLIP2 base patch16-224	embedding.py
<i>Tuple ranking (Section 8, opt-in via <code>apply_tuple_ranking</code>)</i>		
absolute_threshold (α)	0.0	TupleRankingHyperparameters
relative_delta (δ)	0.15	TupleRankingHyperparameters
max_regions_per_event (N)	20	TupleRankingHyperparameters
order_weight (w_o)	0.8	TupleRankingHyperparameters
pooling	"max"	TupleRankingHyperparameters
confidence_gate	"threshold"	TupleRankingHyperparameters
confidence_gate_threshold (τ)	0.5	TupleRankingHyperparameters
max_combinations_per_video	20,000	TupleRankingHyperparameters
max_tuples_per_video	20	TupleRankingHyperparameters